Conversación del proyecto

Calculadora de viajes y combustible en Java · Recopilación cronológica

Este documento reúne la conversación de trabajo sobre el proyecto: la idea inicial, las decisiones sobre el menú, los requisitos de Java, los ejemplos y los fragmentos de código que se han propuesto. Se conserva el contenido técnico y se organiza para que sea fácil consultarlo mientras se desarrolla el proyecto en NetBeans.

Nota: es una recopilación de la conversación disponible en este chat, no una exportación de toda la cuenta ni de otros chats.

---

1. Idea inicial del proyecto

Usuario

Tengo un proyecto que hacer en Java. Mi idea es hacer una calculadora de combustible: introducir kilómetros, consumo y precio de gasolina para calcular el coste de un viaje. Tenemos que cumplir estos requisitos:

```
Scanner (para la entrada de datos)
Random (para generar valores aleatorios)
Estructuras condicionales (if / switch)
Bucles (for / while / do-while)
Gestión de excepciones (try-catch)
```

Asistente

La propuesta fue crear una calculadora de combustible para viajes. El usuario introduciría los kilómetros, el consumo del vehículo (litros cada 100 km) y el precio del combustible por litro. El programa calcularía los litros necesarios, el coste total y el coste por cada 100 km.

Fórmulas principales

```
Litros necesarios = (kilómetros * consumo) / 100
Coste total = litros * precio
```

Ejemplo: para 300 km, un consumo de 6 L/100 km y un precio de 1,60 €/L, se necesitan 18 litros y el coste total es 28,80 €.

2. Primera propuesta de menú

Asistente

Se propuso inicialmente un menú con varias funciones:

```
================================
     CALCULADORA DE COMBUSTIBLE
================================

1. Calcular coste de un viaje
2. Generar viaje aleatorio
3. Comparar costes por distancia
4. Salir

Selecciona una opción:
```

También se propuso que la opción de viaje aleatorio generase kilómetros con Random y que la comparación de distancias utilizase un bucle for.

3. Decisión: dos funciones principales

Usuario

Lo que me gustaría es tener un menú con varias opciones: calcular combustible, calcular coste por persona y salir.

Asistente

Se reorganizó el menú para dar prioridad al cálculo del combustible y al reparto del coste entre las personas que viajan.

```
==================================
       CALCULADORA DE VIAJES
==================================

1. Calcular combustible
2. Calcular coste por persona
3. Salir

Selecciona una opción:
```

Opción 1: calcular combustible

```
Introduce los kilómetros: 300
Introduce el consumo: 6
Introduce el precio por litro: 1.60

--------- RESULTADO ---------

Litros necesarios: 18
Coste total: 28.80 €
```

Opción 2: calcular coste por persona

```
Introduce el coste total: 60 €
¿Cuántas personas viajan?: 4

--------- RESULTADO ---------

Coste total: 60.00 €
Personas: 4
Precio por persona: 15.00 €
```

4. Elección de la calculadora integrada

Asistente

Se plantearon dos posibilidades para calcular el coste por persona:

```
Opción A: Calculadora integrada
Introducir los datos del viaje y el número de personas.
El programa calcula el combustible, el coste total y cuánto paga cada persona.

Opción B: Calculadora independiente
Introducir directamente el coste total y el número de personas.
```

Usuario

Prefiero la opción A, calculadora integrada. Vamos a desarrollar el proyecto paso a paso.

Se eligió la opción A. Por tanto, el cálculo del coste por persona forma parte del mismo cálculo del viaje: kilómetros, consumo, precio por litro y número de personas.

5. Paso 1: crear el proyecto y el menú en NetBeans

Nombre propuesto para el proyecto: **CalculadoraViajes**. Clase principal: **CalculadoraViajes**.

El primer código construye el menú, lee la opción con Scanner, utiliza switch para elegir qué hacer, do-while para repetir el menú y try-catch para controlar una entrada que no sea numérica.

```
import java.util.Scanner;

public class CalculadoraViajes {

    public static void main(String[] args) {

        Scanner entrada = new Scanner(System.in);

        int opcion = 0;

        do {

            System.out.println("================================");
```

```
            System.out.println("      CALCULADORA DE VIAJES");
            System.out.println("==============================");
            System.out.println("1. Calcular combustible");
            System.out.println("2. Generar viaje aleatorio");
            System.out.println("3. Salir");
            System.out.println("==============================");
            System.out.print("Selecciona una opcion: ");

            try {

                opcion = entrada.nextInt();

                switch (opcion) {

                    case 1:
                        System.out.println("Has seleccionado calcular combustible.");
                        break;

                    case 2:
                        System.out.println("Has seleccionado viaje aleatorio.");
                        break;

                    case 3:
                        System.out.println("Gracias por utilizar la calculadora. Buen viaje!");
                        break;

                    default:
                        System.out.println("Opcion incorrecta. Intentalo de nuevo.");
                        break;
                }

            } catch (Exception e) {

                System.out.println("Error: debes introducir un numero.");
                entrada.nextLine();
            }

        } while (opcion != 3);

        entrada.close();
    }
}
```

Pruebas propuestas

```
1 -> muestra el mensaje de calcular combustible.
2 -> muestra el mensaje de viaje aleatorio.
5 -> muestra que la opción es incorrecta.
hola -> muestra un mensaje de error.
3 -> sale del programa.
```

Importante: en este primer borrador, el case 1 y el case 2 solo mostraban mensajes. Las operaciones se añadieron después.

6. Paso 2: desarrollar el viaje aleatorio con Random

Usuario

Perfecto, ahora nos faltaría la opción 2, no el de viaje aleatorio.

Asistente

Se aclaró que la opción 2 podía generar automáticamente los datos de un viaje para demostrar el uso de Random.

Primero se añadió la importación y la creación del objeto:

```
import java.util.Random;

// Dentro de main, debajo de Scanner:
Random aleatorio = new Random();
```

Después se propuso sustituir el case 2 por este fragmento:

```
case 2:

    System.out.println("GENERANDO VIAJE ALEATORIO...");

    int kilometros = aleatorio.nextInt(751) + 50;
    double consumo = aleatorio.nextInt(7) + 4;
```

```
    double precio = aleatorio.nextInt(41) / 100.0 + 1.40;
    int personas = aleatorio.nextInt(5) + 1;

    double litros = (kilometros * consumo) / 100;
    double coste = litros * precio;
    double costePersona = coste / personas;

    System.out.println("------------------------------");
    System.out.println("       VIAJE ALEATORIO");
    System.out.println("------------------------------");

    System.out.println("Kilometros: " + kilometros + " km");
    System.out.println("Consumo: " + consumo + " L/100 km");
    System.out.println("Precio combustible: " + precio + " euros");
    System.out.println("Personas: " + personas);

    System.out.println("------------------------------");

    System.out.printf("Litros necesarios: %.2f L%n", litros);
    System.out.printf("Coste total: %.2f euros%n", coste);
    System.out.printf("Coste por persona: %.2f euros%n", costePersona);

    System.out.println("------------------------------");

    break;
```

Rangos de los valores aleatorios

```
aleatorio.nextInt(751) + 50
Genera kilómetros entre 50 y 800.

aleatorio.nextInt(7) + 4
Genera un consumo entre 4 y 10 L/100 km.

aleatorio.nextInt(41) / 100.0 + 1.40
Genera un precio entre 1,40 € y 1,80 €.

aleatorio.nextInt(5) + 1
Genera entre 1 y 5 personas.
```

La opción aleatoria es secundaria: sirve para generar un ejemplo automático y utilizar Random.

7. Problema detectado: permitir kilómetros negativos

Usuario

He visto un fallo: en calcular combustible, en introducir los kilómetros puedo poner un valor negativo y los datos como litros necesarios y coste total salen en negativo.

Asistente

Se propuso validar los datos con if antes de realizar los cálculos. La condición general era:

```
if (kilometros <= 0 || consumo <= 0 || precio <= 0 || personas <= 0) {

    System.out.println("Error: Los valores deben ser mayores que 0.");

} else {

    litros = (kilometros * consumo) / 100;
    coste = litros * precio;
    costePersona = coste / personas;

    System.out.println("Litros necesarios: " + litros);
    System.out.println("Coste total: " + coste + " euros");
    System.out.println("Coste por persona: " + costePersona + " euros");
}
```

Esta primera solución detectaba los valores incorrectos después de haber introducido los datos. El usuario explicó que quería una validación inmediata, dato por dato.

8. Validar cada dato justo después de introducirlo

Usuario

No quiero que el programa espere a que introduzcas todos los datos para mostrar un error. Quiero que compruebe cada dato justo después de introducirlo y, si es negativo o cero, vuelva a preguntar hasta que introduzcas un número positivo.

Se cambió la solución: ahora cada dato tiene su propio bucle while. El programa no pasa a la siguiente pregunta hasta que el valor actual sea mayor que cero.

Ejemplo de cómo debe comportarse

```
Introduce los kilómetros del viaje: -1000
Error: Los kilómetros deben ser mayores que 0.
Introduce los kilómetros del viaje: 0
Error: Los kilómetros deben ser mayores que 0.
Introduce los kilómetros del viaje: 300

Introduce el consumo del coche (L/100 km):
```

Código completo propuesto para el case 1

```
case 1:

    double kilometros;
    double consumo;
    double precio;
    int personas;
    double litros;
    double coste;
    double costePersona;

    System.out.println("CALCULAR COMBUSTIBLE");

    // Introducir kilometros
    System.out.print("Introduce los kilometros del viaje: ");
    kilometros = entrada.nextDouble();

    while (kilometros <= 0) {
        System.out.println("Error: Los kilometros deben ser mayores que 0.");
        System.out.print("Introduce los kilometros del viaje: ");
        kilometros = entrada.nextDouble();
    }

    // Introducir consumo
    System.out.print("Introduce el consumo del coche (L/100 km): ");
    consumo = entrada.nextDouble();

    while (consumo <= 0) {
        System.out.println("Error: El consumo debe ser mayor que 0.");
        System.out.print("Introduce el consumo del coche (L/100 km): ");
        consumo = entrada.nextDouble();
    }

    // Introducir precio
    System.out.print("Introduce el precio de gasolina por litro: ");
    precio = entrada.nextDouble();

    while (precio <= 0) {
        System.out.println("Error: El precio debe ser mayor que 0.");
        System.out.print("Introduce el precio de gasolina por litro: ");
        precio = entrada.nextDouble();
    }

    // Introducir personas
    System.out.print("Introduce el numero de personas: ");
    personas = entrada.nextInt();

    while (personas <= 0) {
        System.out.println("Error: Debe haber al menos una persona.");
        System.out.print("Introduce el numero de personas: ");
        personas = entrada.nextInt();
    }

    // Calculos
    litros = (kilometros * consumo) / 100;
    coste = litros * precio;
    costePersona = coste / personas;

    // Resultados
    System.out.println("------------------------------");
```

```
        System.out.println("       RESULTADO DEL VIAJE");
        System.out.println("------------------------------");

        System.out.println("Kilometros: " + kilometros + " km");
        System.out.println("Consumo: " + consumo + " L/100 km");
        System.out.println("Precio gasolina: " + precio + " euros");
        System.out.println("Personas: " + personas);

        System.out.printf("Litros necesarios: %.2f L%n", litros);
        System.out.printf("Coste total: %.2f euros%n", coste);
        System.out.printf("Coste por persona: %.2f euros%n", costePersona);

        System.out.println("------------------------------");

        break;
```

Qué valida cada while: kilometros <= 0, consumo <= 0, precio <= 0 y personas <= 0. En todos los casos se vuelve a preguntar inmediatamente. Las personas deben ser un número entero positivo.

Límite de esta versión: estos while controlan cero y números negativos, pero todavía no controlan letras introducidas donde se espera un número. Para eso habrá que integrar try-catch también en las entradas de datos.

9. Decisión sobre mantener la opción aleatoria

Usuario

Creo que el apartado de calcular aleatorio, es decir, generar viaje aleatorio, lo voy a quitar. No sé, no tiene uso. El apartado uno es más importante. Creo que el apartado dos no lo pondría.

Asistente

Se estuvo de acuerdo en que la calculadora debía centrarse en el cálculo real, pero se recordó que Random es un requisito obligatorio. Se propuso eliminar la opción aleatoria del menú y utilizar Random en una función secundaria dentro de la calculadora.

Usuario

Ah, es verdad, que había que usar el Random también. Sí, sí.

La decisión final fue mantener la opción de viaje aleatorio en el menú para cumplir el requisito, pero dar prioridad al cálculo manual del combustible y del coste por persona.

10. Estructura y requisitos que debe cumplir el proyecto

Menú acordado

```
================================
       CALCULADORA DE VIAJES
================================

1. Calcular combustible y coste por persona
2. Generar viaje aleatorio
3. Salir

Selecciona una opción:
```

Lista de requisitos

```
Scanner
Leer los datos que introduce el usuario.

Random
Generar automáticamente los datos de un viaje de ejemplo.

if / else
Comprobar si los datos son válidos y tomar decisiones.

switch
Elegir la opción del menú.

for
Se debe incorporar en alguna funcionalidad útil del programa.
```

```
while
Volver a preguntar un dato hasta que sea correcto.

do-while
Repetir el menú hasta que el usuario elija salir.

try-catch
Controlar errores de entrada, por ejemplo, cuando se escriben letras
en lugar de números.
```

La opción for todavía está pendiente de integrar en una funcionalidad que tenga sentido. También queda reforzar try-catch para las entradas numéricas de las opciones, no solo para la selección del menú.

11. Estado actual y próximos pasos

Ya se ha trabajado:

```
[x] Idea del proyecto.
[x] Menú principal inicial.
[x] Scanner para leer datos.
[x] switch para seleccionar opciones.
[x] do-while para repetir el menú.
[x] Random para generar un viaje de ejemplo.
[x] Fórmulas de litros, coste total y coste por persona.
[x] Validación inmediata de números positivos con while.
```

Pendiente:

```
[ ] Integrar el código de la opción 1 en el programa completo.
[ ] Revisar y completar la opción 2 con Random.
[ ] Añadir try-catch a las entradas de datos para controlar letras.
[ ] Añadir una funcionalidad útil que utilice for.
[ ] Revisar el uso de if / else.
[ ] Probar el programa completo y corregir errores.
[ ] Comprobar que todos los requisitos del profesor están presentes.
```

Recomendación para continuar: seguir trabajando en NetBeans por pasos pequeños, probar cada cambio y mantener el código al nivel de Java que se está estudiando en clase.